# Review Exercises

### Preparations

### Loading packages

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf
import itertools
```

### Loading dataset

The dataset is the hotels-europe dataset.

```
ROOT = r'C:\Users\PC_DS_ECON_5\Desktop\data-analytics-python'
hotels_europe = pd.read_parquet(ROOT + '/data/hotels_europe_restr_df.parquet')
```

### Variables

```
hotels_europe.columns
```

```
Index(['hotel_id', 'city', 'distance', 'stars', 'rating', 'country',
       'accommodation_type', 'price', 'holiday', 'yearmonth'],
      dtype='str')
```

| Variable | Description |
|---|---|
| `hotel_id` | Hotel identifier. |
| `yearmonth` | Month of observation. |
| `city` | City in which the hotel is located. |
| `distance` | Distance between the hotel and the city center (km). |
| `stars` | Hotel star rating. |
| `rating` | Average customer rating. |
| `country` | Country in which the hotel is located. |
| `accommodation_type` | Type of accommodation offered by the hotel. |
| `price` | Hotel price per night (USD). |
| `holiday` | Indicator for whether the observation refers to a holiday period. |

### Ex.1. Inspecting the variable `stars`

a. What does the `value_counts()` method do in each of the following cases?

b. What does the `sort_values()` method do?

c. What does the `pd.merge()` method do in the current case?

d. In the dataframe `df3`, what does the column `count` correspond to?

e. In the dataframe `df3`, what does the column `proportion` correspond to?

```
df0= hotels_europe.copy()
df1= (
    df0['stars']
        .value_counts(normalize=False)
        .reset_index()
        .sort_values(by='stars', ascending=True)
        .reset_index(drop=True)
        .copy()
)
print('df1 columns:', df1.columns.to_list())
df2= (
    df0['stars']
        .value_counts(normalize=True)
        .reset_index()
        .sort_values(by='stars', ascending=True)
        .reset_index(drop=True)
        .copy()
)
print('df2 columns:', df2.columns.to_list())
df3= pd.merge(
    left=df1,
    right=df2,
    on='stars',
    how='outer'
)
df3
```

```
df1 columns: ['stars', 'count']
df2 columns: ['stars', 'proportion']
```

| | stars | count | proportion |
|---|---|---|---|
| 0 | 1.0 | 613 | 0.022651 |
| 1 | 1.5 | 45 | 0.001663 |
| 2 | 2.0 | 3391 | 0.1253 |
| 3 | 2.5 | 856 | 0.03163 |
| 4 | 3.0 | 9683 | 0.357795 |
| 5 | 3.5 | 2049 | 0.075712 |
| 6 | 4.0 | 8378 | 0.309574 |
| 7 | 4.5 | 355 | 0.013118 |
| 8 | 5.0 | 1693 | 0.062558 |

### Ex. 2. Unit of observation

Based on the code and the corresponding output below, what is the unit of observation in the `hotels_europe` dataframe here? Briefly explain why.

i. The unit of observation is `hotel_id`.

ii. The unit of observation is `hotel_id` × `yearmonth`

iii. The unit of observation is `hotel_id` × `yearmonth` × `holiday`

```
df0= hotels_europe.copy()
df1= (
    df0
        .value_counts(subset=['hotel_id'])
        .reset_index()
        .rename(columns={'count': 'n_obs'})
        .value_counts(subset=['n_obs'])
        .reset_index()
        .copy()
)
print('1:\n', df1, '\n', sep='')
df2= (
    df0
        .value_counts(subset=['hotel_id', 'yearmonth'])
        .reset_index()
        .rename(columns={'count': 'n_obs'})
        .value_counts(subset=['n_obs'])
        .reset_index()
        .copy()
)
print('2:\n', df2, '\n', sep='')
df3= (
    df0
        .value_counts(subset=['hotel_id', 'yearmonth', 'holiday'])
        .reset_index()
        .rename(columns={'count': 'n_obs'})
        .value_counts(subset=['n_obs'])
        .reset_index()
        .copy()
)
print('3:\n', df3, '\n', sep='')
```

```
1:
   n_obs  count
0      2  13898
1      1   5446

2:
   n_obs  count
0      1  33242

3:
```

```
   n_obs  count
0      1  33242
```

### Ex. 3. Transforming the variable `stars`

a. Explain the purpose of the code below.

b. What is the new variable `stars_cat`?

c. Why are `drop_duplicates()` and `dropna()` used?

```
df0= hotels_europe.copy()
print('stars variable datatype:', df0['stars'].dtype)

stars_unique_values= (
    df0['stars']
        .drop_duplicates()
        .dropna()
        .sort_values(ascending=True)
        .to_list()
        .copy()
)
print('stars_unique_values:', stars_unique_values)
df0['stars_cat'] = pd.Categorical(
    values=df0['stars'],
    categories=stars_unique_values,
    ordered=True
)

hotels_europe= df0.copy()
```

```
stars variable datatype: Float64
stars_unique_values: [1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0]
```

### Ex. 4. Inspecting the variable `distance`

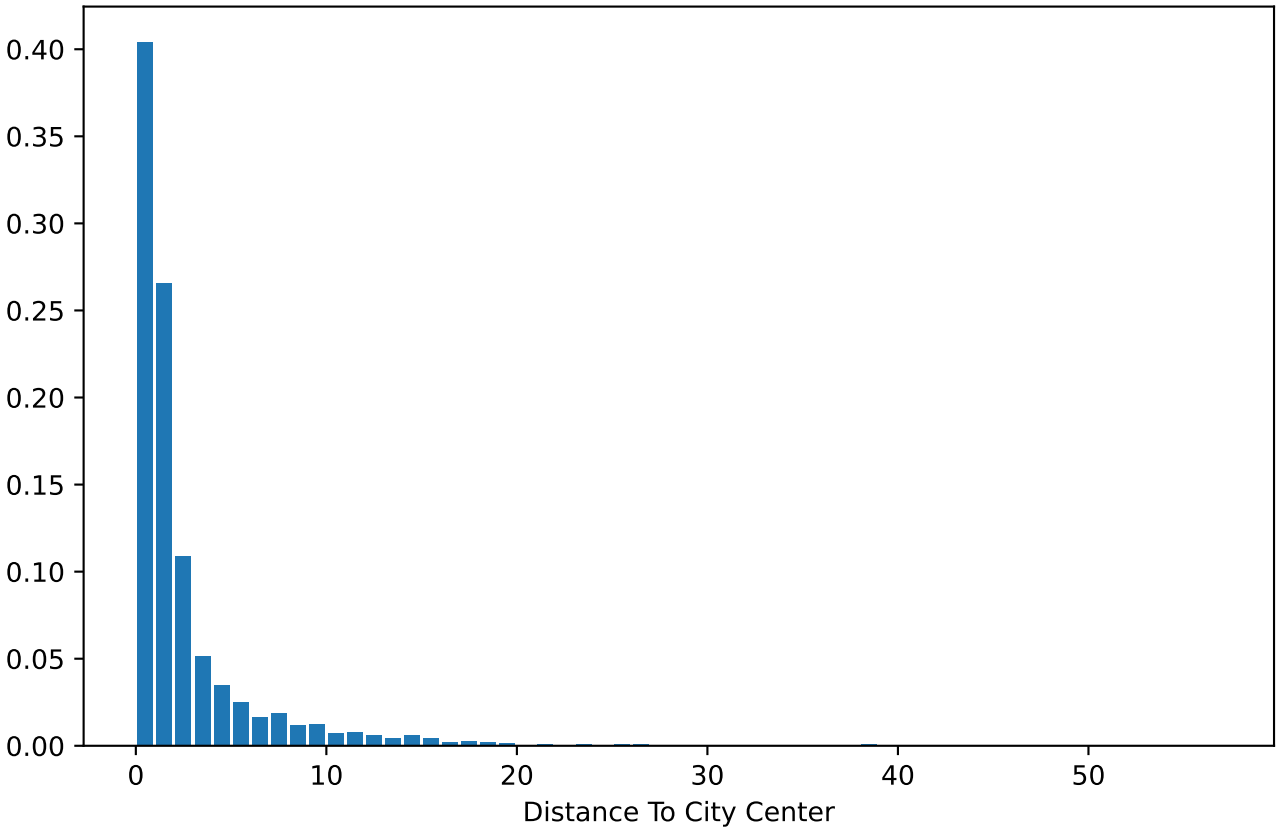
a. In the code below, what is the purpose of the line with `.loc`?

b. What kind of plot does the code below produce?

c. How would you label the vertical (y-)axis?

d. Is the distribution of `distance` symmetric? Would you expect the median of `distance` or the mean of `distance` to be higher?

```
binwidth = 1
df0= hotels_europe.copy()

df0 = df0.loc[df0['yearmonth'].eq('2017-11-01'),:].copy()

max_distance = df0['distance'].max()
max_upper_bound = np.ceil(max_distance)

df0['distance_interval']= pd.cut(
    df0['distance'],
    bins=np.arange(
        start=0,
        stop=max_upper_bound + binwidth,
        step=binwidth),
    include_lowest=True
)

df0['distance_interval_mid'] = (
    df0['distance_interval']
        .map(lambda x: x.mid)
)

df1= (
    df0
        .value_counts(subset=['distance_interval_mid'], normalize=True)
        .reset_index()
        .sort_values(by=['distance_interval_mid'], ascending=True)
        .reset_index(drop=True)
)

fig, ax = plt.subplots()
ax.bar(
    x=df1['distance_interval_mid'],
    height=df1['proportion'])
ax.set_xlabel('Distance To City Center')
plt.show()
```


0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
10
20
30
40
50
Distance To City Center

### Ex. 5. Defining distance categories

Briefly explain the purpose of the following code.

```
binwidth = 1
df0= hotels_europe.copy()
max_upper_bound = np.ceil(df0['distance'].max() / 10) * 10
interval_bounds = [0, 1, 2, 3, 4, 6, 8, 10, 20, max_upper_bound]

df0['distance_cat_int']= pd.cut(
    df0['distance'],
    bins=interval_bounds,
    include_lowest=True
)

print('distance_cat_int:\n', repr(df0['distance_cat_int'].dtype))

distance_cat_list = (
    df0['distance_cat_int']
        .dropna()
        .drop_duplicates()
        .sort_values()
        .to_list()
)

distance_cat_dict = {
    iv: (
        f'({int(iv.left)}, {int(iv.right)}]'
        if int(iv.left) > 0
        else f'[{int(iv.left)}, {int(iv.right)}]'
    )
    for iv in distance_cat_list
}

df0['distance_cat'] = (
    df0['distance_cat_int']
        .map(lambda x: distance_cat_dict[x])
)

distance_cat_unique = [cat for iv, cat in distance_cat_dict.items()]

df0['distance_cat'] = pd.Categorical(
    values=df0['distance_cat'],
    categories=distance_cat_unique,
    ordered=True
)

print(
    'distance_cat:\n',
    repr(df0['distance_cat'].dtype)
)
```

```
hotels_europe = df0.copy()
```

```
distance_cat_int:
 CategoricalDtype(categories=[(-0.001, 1.0],    (1.0, 2.0],    (2.0, 3.0],    (3.0, 4.0],
                      (4.0, 6.0],    (6.0, 8.0],   (8.0, 10.0],  (10.0, 20.0],
                     (20.0, 60.0]],
, ordered=True, categories_dtype=interval[float64, right])
distance_cat:
 CategoricalDtype(categories=['[0, 1]', '(1, 2]', '(2, 3]', '(3, 4]', '(4, 6]', '(6, 8]',
                  '(8, 10]', '(10, 20]', '(20, 60]'],
, ordered=True, categories_dtype=str)
```

### Ex. 6. Descriptive statistics

a. In the code below, what is the purpose of the line with `.loc`?

b. For each of the statistics `statistic[1]`, `statistic[2]`, `statistic[3]`, `statistic[4]`:

   i. Explain what the statistic corresponds to.
   ii. Whether it is a measure of central tendency or a measure of dispersion.

```
df0 = hotels_europe.copy()
df0 = df0.loc[df0['price'].notna(),:].copy()

statistic = {}

statistic[1] = (
    df0['price'].sum()/df0['price'].shape[0]
)

statistic[2] = (
    ((df0['price'] - df0['price'].mean()) ** 2)
        .mean()
)
statistic[3] = (
    np.sqrt(
        ((df0['price'] - df0['price'].mean()) ** 2)
            .mean()
    )
)

statistic[4] = (
    df0['price'].quantile(0.5)
)

print(
    f'statistic 1: {statistic[1]:.1f}\n',
    f'statistic 2: {statistic[2]:.1f}\n',
    f'statistic 3: {statistic[3]:.1f}\n',
    f'statistic 4: {statistic[4]:.1f}',
    sep=''
)
```

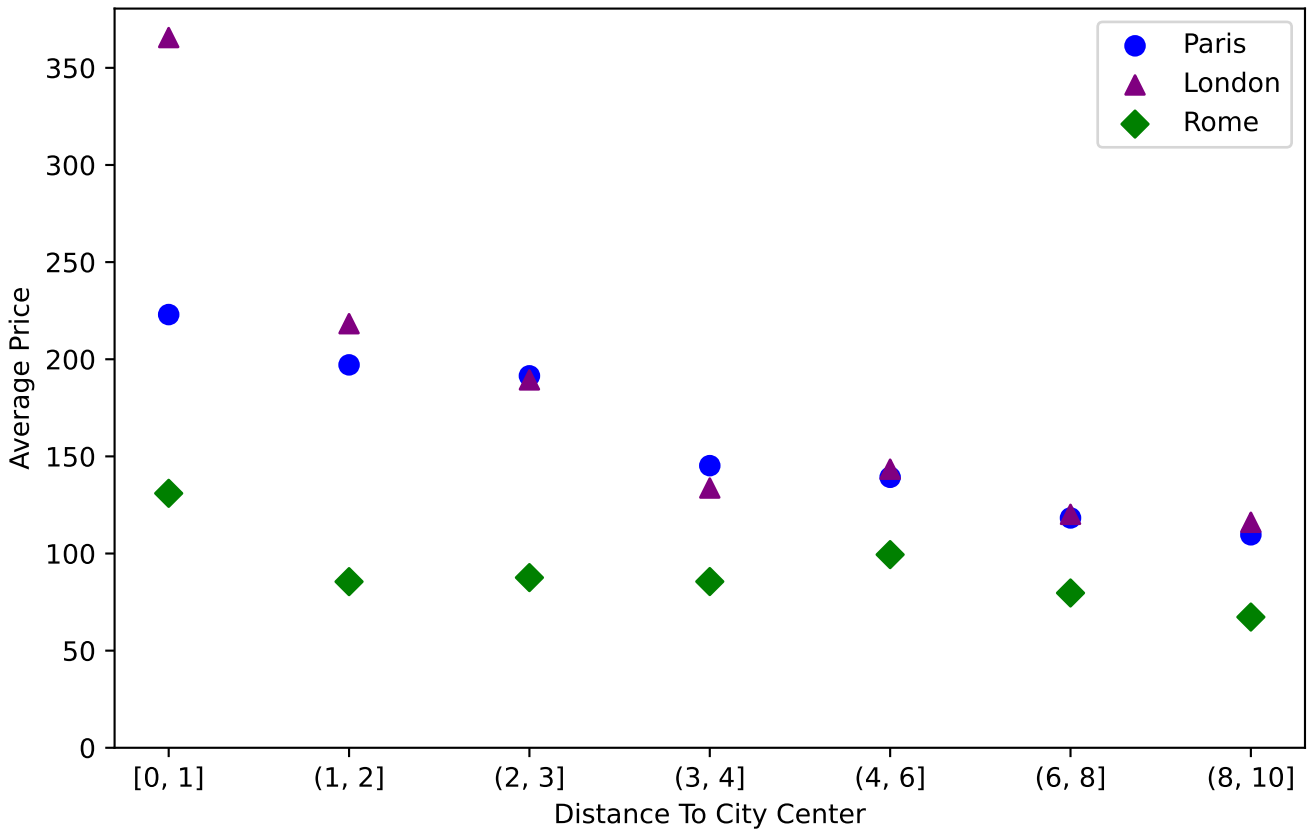
```
statistic 1: 118.2
statistic 2: 11434.2
statistic 3: 106.9
statistic 4: 93.0
```

**Ex. 7. Relationship between price and distance in three major cities**

a. Explain the purpose of the use of the two `.loc` in the code below.

b. Based on the code and the output below, holding city constant, does price appear to be mean independent from distance? Formally, does it appear to be the case that $\mathbb{E}[price \mid distance, city] \approx \mathbb{E}[price \mid city]$ ? Briefly justify your answer.

```
df0 = hotels_europe.copy()
cities = ['Paris', 'London', 'Rome']
colours = ['blue', 'purple', 'green']
markers = ['o', '^', 'D']
df0 = df0.loc[df0['yearmonth'].eq('2017-11-01') &
              df0['city'].isin(cities) &
              df0['distance'].le(10) &
              df0['distance'].notna() &
              df0['price'].notna(),:].copy()

df1 = (
    df0
        .groupby(by=['distance_cat', 'city'])
        .agg(mean_price_by_dist_and_city = ('price', 'mean'))
        .reset_index()
        .sort_values(by=['city', 'distance_cat'])
        .reset_index(drop=True)
        .copy()
)

fig, ax = plt.subplots()
for i, city in enumerate(cities):
    df2 = df1.loc[df1['city'].eq(city),:].copy()
    ax.scatter(
        x=df2['distance_cat'].astype(str),
        y=df2['mean_price_by_dist_and_city'],
        label=city,
        marker=markers[i],
        color=colours[i],
        s=50
    )
ax.set_ylabel('Average Price')
ax.set_xlabel('Distance To City Center')
ax.set_ylim(bottom=0)
ax.legend()
```




### Saving selected data

Saving the selected dataset as `hotels_selected`.

```
hotels_selected = df0.copy()
```

**Ex. 8. Linear regression of price on distance for three major cities**

Consider the following linear regression implemented in the code below:

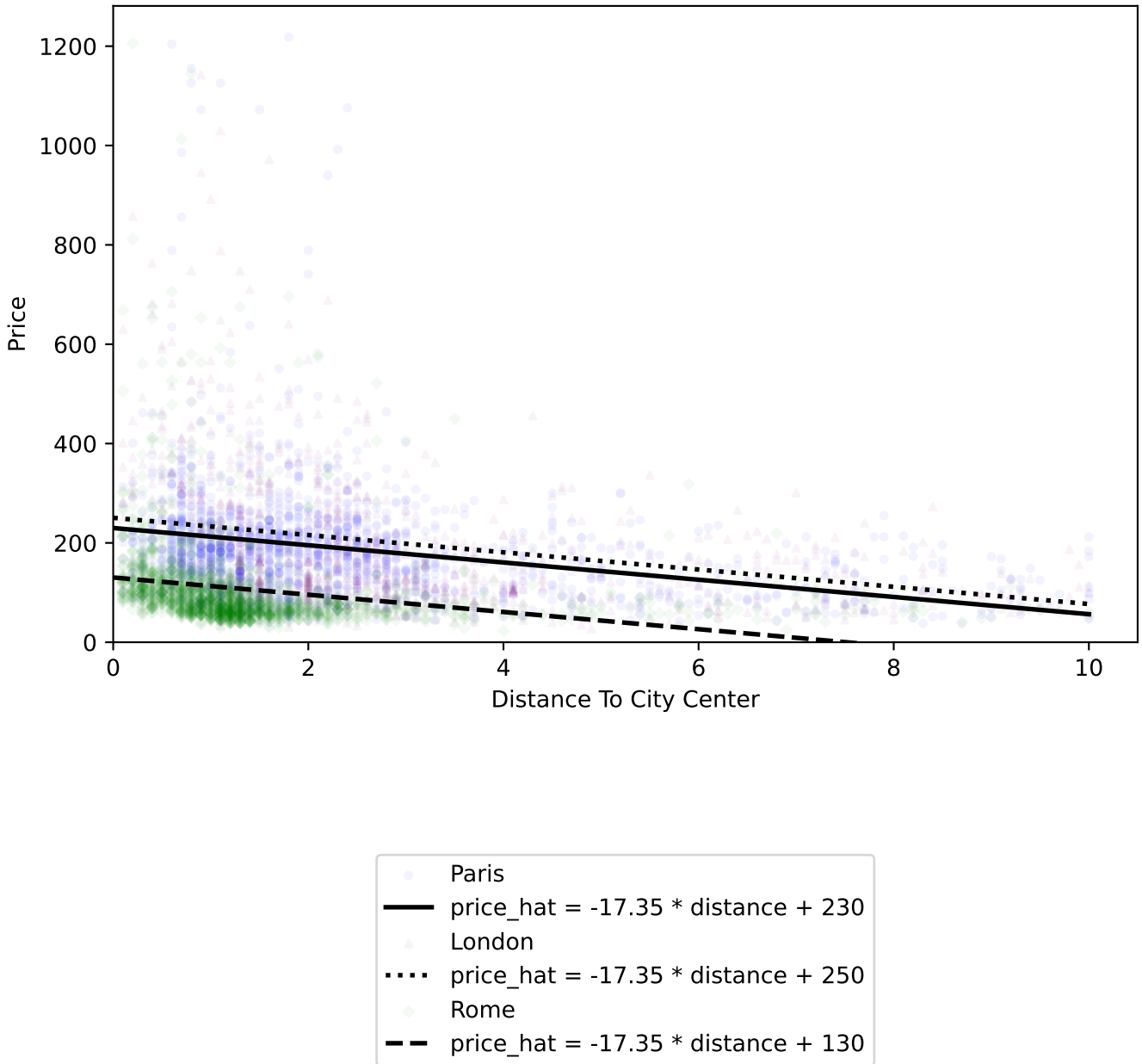
$$\begin{aligned} price &= \beta_1 \cdot distance + \beta_{0,London} \cdot \mathbb{1}[city = London] + \\ &+ \beta_{0,Paris} \cdot \mathbb{1}[city = Paris] + \beta_{0,Rome} \cdot \mathbb{1}[city = Rome] \end{aligned}$$

What is the interpretation of each of the coefficients $\beta_1, \beta_{0,London}, \beta_{0,Paris}, \beta_{0,Rome}$ ?

```
df0 = hotels_selected.copy()
cities = ['Paris', 'London', 'Rome']
colours = ['blue', 'purple', 'green']
markers = ['o', '^', 'D']
styles = ['solid', 'dotted', 'dashed']
df0 = (
    df0[['price', 'distance', 'city', 'rating']]
        .dropna()
        .reset_index(drop=True)
        .copy()
)

reg = smf.ols(formula='price ~ 0 + distance + city', data=df0).fit()
print(reg.params)
beta_1 = reg.params['distance']

fig, ax = plt.subplots()
for i, city in enumerate(cities):
    df2 = df0.loc[df0['city'].eq(city),:].copy()
    intercept = reg.params['city[' + city + ']']
    ax.scatter(
        x=df2['distance'],
        y=df2['price'],
        label=city,
        marker=markers[i],
        color=colours[i],
        s=10,
        alpha=0.05
    )
    distance_values = np.linspace(start =0, stop=10, num= 1000)
    price_hat = intercept + beta_1 * distance_values
    ax.plot(
        distance_values,
        price_hat,
        color='black',
        label=f'price_hat = {reg.params['distance']:.2f} * distance + {intercept:.0f}',
        linestyle=styles[i],
        linewidth= 2
    )

ax.set_ylabel('Price')
ax.set_xlabel('Distance To City Center')
```

```
ax.set_ylim(bottom=0)
ax.set_xlim(left=0)
ax.legend(
    bbox_to_anchor=(0.5,-0.5),
    loc='center'
)
```

```
city[London]     250.469641
city[Paris]      229.993839
city[Rome]       130.401668
distance         -17.351616
dtype: float64
```



### Centering / demeaning the variable rating

The variable `rating_centered` is defined as `rating` minus the mean of `rating`:

$$rating\_centered_i = rating_i - \overline{rating}$$

```
df0 = hotels_selected.copy()
df0['rating_centered'] = df0['rating'] - df0['rating'].mean()
hotels_selected = df0.copy()
```

### Ex. 9. Adding (centered) hotel ratings to the linear regression of price on distance for three major cities

Consider the following three regressions implemented by the code below.

Explain what the two identical values printed at the end of the code represent.

```
df0 = hotels_selected.copy()
cities = ['Paris', 'London', 'Rome']

df0 = (
    df0[['price', 'distance', 'city', 'rating_centered']]
        .dropna()
        .reset_index(drop=True)
        .copy()
)

include_rating = smf.ols(
    formula='price ~ distance + rating_centered + city',
    data=df0
    ).fit()
omit_rating = smf.ols(
    formula='price ~ distance + city',
    data=df0
    ).fit()
auxiliary = smf.ols(
    formula='rating_centered ~ distance + city',
    data=df0
    ).fit()

print(
    'Include rating:\n',
    include_rating.params[['distance', 'rating_centered']],
    '\n\n',
    'Omit rating:\n',
    omit_rating.params[['distance']],
    '\n\n',
    'Auxiliary:\n',
    auxiliary.params[['distance']],
    '\n\n',
    'Difference:\n',
    omit_rating.params['distance'] - include_rating.params['distance'],
    '\n\n',
    'Product:\n',
    include_rating.params['rating_centered'] * auxiliary.params['distance'],
    sep=''
)
```

```
Include rating:
distance          -13.565516
rating_centered    67.620771
dtype: float64
```

```
Omit rating:
distance   -17.351616
dtype: float64

Auxiliary:
distance   -0.05599
dtype: float64

Difference:
-3.786099518130273

Product:
-3.7860995181300505
```

**Ex. 10. Including distance as categorical variable in the linear regression of price on distance for three major cities**

Consider the following linear regression:

$$\begin{aligned} price &= \beta_0 + \sum_{j \in DistanceCategories} \beta_j \cdot \mathbb{1}[DistanceCategory = j] + \\ &+ \gamma \cdot rating\_centered + \delta_{Paris} \cdot \mathbb{1}[city = Paris] + \delta_{Rome} \cdot \mathbb{1}[city = Rome] \end{aligned}$$

a. What is the interpretation of the intercept coefficient $\hat{\beta}_0$ / `Intercept`?

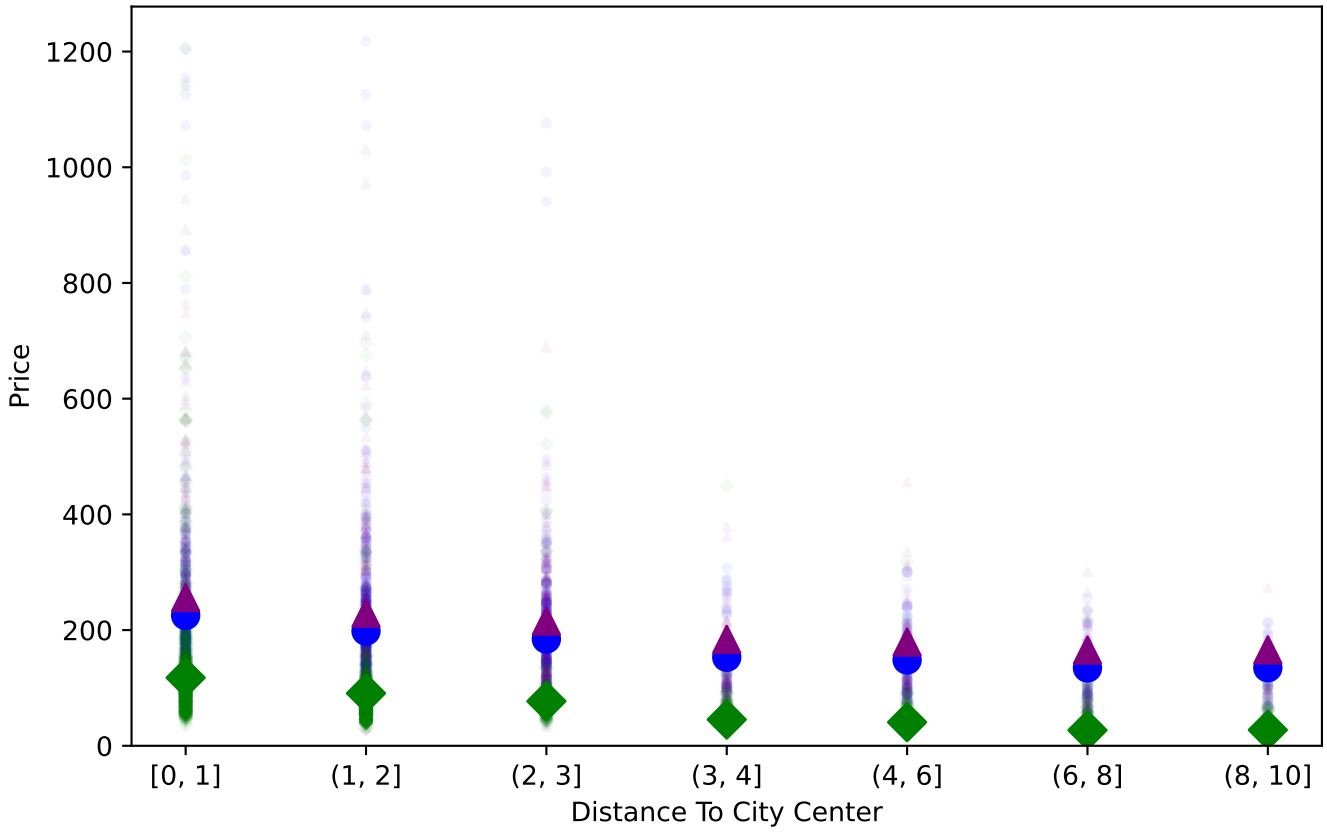
b. What is the interpretation of coefficient $\hat{\beta}_{(3,4]}$ / `distance_cat[T.(3, 4]]`?

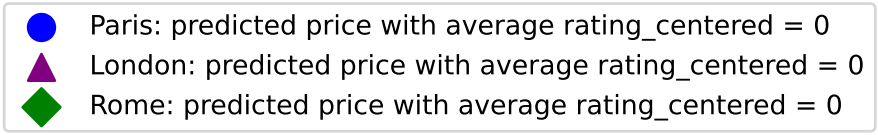
c. What is the interpretation of coefficient $\hat{\delta}_{Rome}$ / `city[T.Rome]`?

```
df0 = hotels_selected.copy()
cities = ['Paris', 'London', 'Rome']
markers = ['o', '^', 'D']

df0['distance_cat'] = df0['distance_cat'].cat.remove_unused_categories()
df0 = (
    df0[['price', 'distance_cat', 'city', 'rating_centered']]
        .dropna()
        .reset_index(drop=True)
        .copy()
)

reg = smf.ols(
    formula='price ~ distance_cat + rating_centered + city',
    data=df0
    ).fit()

print(reg.params)

df1 = df0.copy()
df1['rating_centered'] = 0
df1['price_hat'] = reg.predict(df1)

fig, ax = plt.subplots()
for i, city in enumerate(cities):
    df2 = df1.loc[df1['city'].eq(city),:].copy()
    df2 = (
        df2
            .sort_values(by=['distance_cat'])
            .reset_index(drop=True)
            .copy()
    )
    ax.scatter(
        x=df2['distance_cat'],
        y=df2['price'],
        marker=markers[i],
        color=colours[i],
```

```
        s=10,
        alpha=0.05
    )
for i, city in enumerate(cities):
    df2 = df1.loc[df1['city'].eq(city),:].copy()
    df2 = (
        df2[['distance_cat', 'price_hat', 'city']]
        .drop_duplicates()
        .sort_values(by=['distance_cat'])
        .reset_index(drop=True)
        .copy()
    )
    ax.scatter(
        df2['distance_cat'],
        df2['price_hat'],
        label=city + ': predicted price with average rating_centered = 0',
        color=colours[i],
        s=100,
        marker=markers[i]
    )

ax.set_ylabel('Price')
ax.set_xlabel('Distance To City Center')
ax.set_ylim(bottom=0)
ax.legend(
    bbox_to_anchor=(0.5,-0.5),
    loc='center'
)
```

```
Intercept                  256.088672
distance_cat[T.(1, 2]]     -26.941947
distance_cat[T.(2, 3]]     -40.666527
distance_cat[T.(3, 4]]     -72.282671
distance_cat[T.(4, 6]]     -76.892858
distance_cat[T.(6, 8]]     -90.705536
distance_cat[T.(8, 10]]    -90.291501
city[T.Paris]              -30.850610
city[T.Rome]              -138.452249
rating_centered             65.470361
dtype: float64
```


1200
1000
800
600
400
200
0
Price
[0, 1]
(1, 2]
(2, 3]
(3, 4]
(4, 6]
(6, 8]
(8, 10]
Distance To City Center


Paris: predicted price with average rating_centered = 0
London: predicted price with average rating_centered = 0
Rome: predicted price with average rating_centered = 0

### Ex. 11. Confidence interval for the price-distance relationship

The code below: - simulates the bootstrap distribution of the coefficient estimate $\widehat{\beta}_{(3,4]}$ / `distance_cat[T.(3, 4]]`, - plots the simulated bootstrap distribution, - and calculates selected quantiles of this distribution.

a. Based on the bootstrap percentile method, how would you construct a symmetric 95% confidence interval for the regression coefficient? Which quantile would be the lower bound and which quantile would be the upper bound?

b. What is the reason behind constructing a confidence interval instead of simply interpreting the estimated coefficient as the true regression coefficient?

c. Which familiar probability distribution does the shape of the bootstrap distribution resemble?

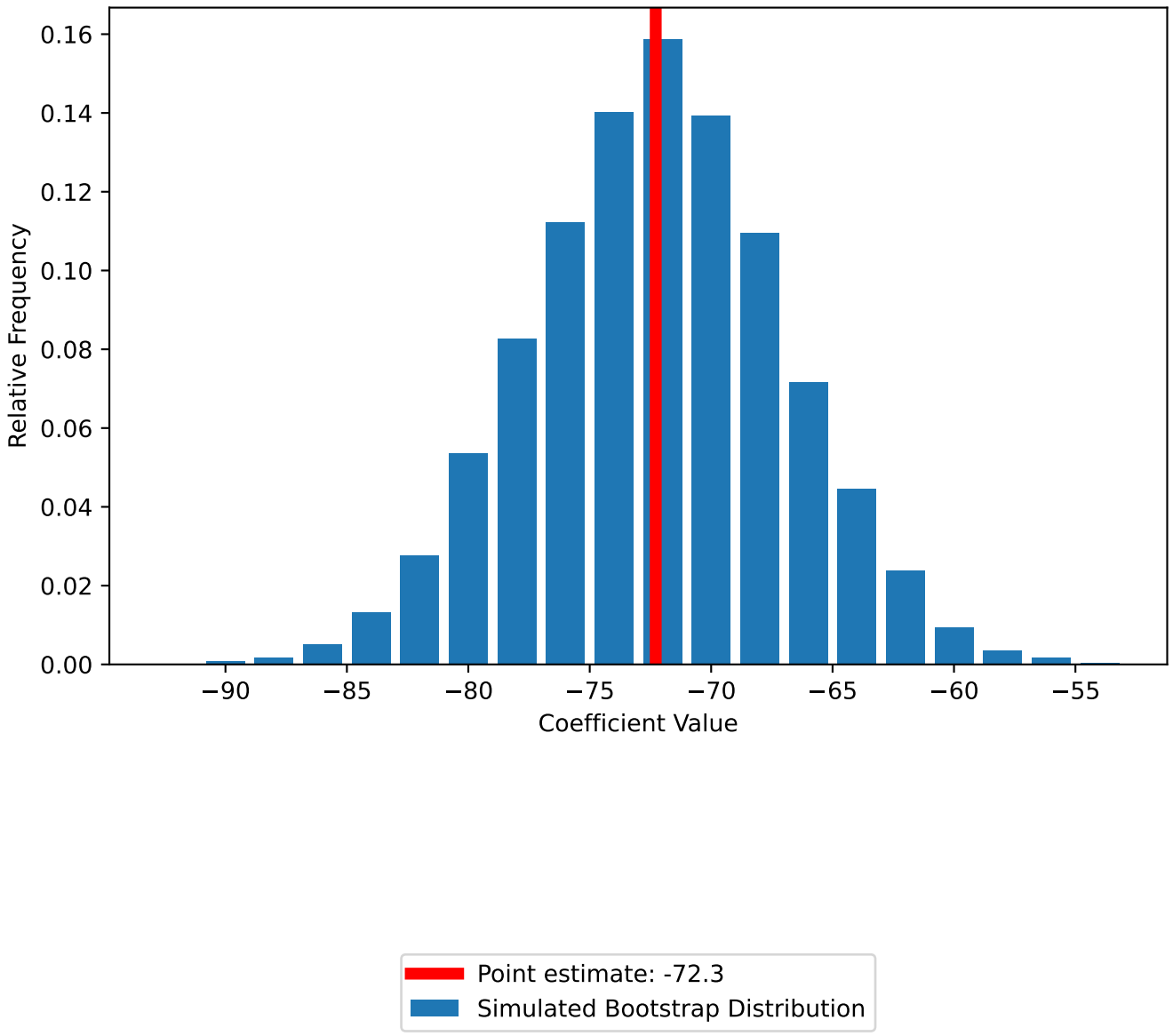
d. What would be another approach to constructing a confidence interval for the regression coefficient? Which theorem can be used to justify this approach? What does the theorem say?

```
df0= hotels_selected.copy()

df0= (
    df0[['price', 'distance_cat', 'city', 'rating_centered']]
        .dropna()
        .reset_index(drop=True)
        .copy()
)

reg= smf.ols(
    formula='price ~ distance_cat + rating_centered + city',
    data=df0
    ).fit()

point_estimate= reg.params['distance_cat[T.(3, 4]]']

rng= np.random.default_rng(946372)

R= 10000
N= df0.shape[0]
beta_sim= np.empty(R)
for i in range(R):
    bootstrap_indices= rng.choice(
        a=N,
        size=N,
        replace=True
    )

    dfr= df0.iloc[bootstrap_indices]

    reg_bootstrap= smf.ols(
        formula='price ~ distance_cat + rating_centered + city',
        data=dfr
        ).fit()
    beta_sim[i]= reg_bootstrap.params['distance_cat[T.(3, 4]]']
```

```
dfs = pd.DataFrame({'beta_sim': beta_sim})
beta_sim_min = dfs['beta_sim'].min()
beta_sim_max = dfs['beta_sim'].max()
lower_bound_min = np.floor(beta_sim_min)
upper_bound_max = np.ceil(beta_sim_max)
binwidth = 2
bins = np.arange(
    start=lower_bound_min,
    stop=upper_bound_max + binwidth,
    step=binwidth
)

dfs['beta_sim_int'] = pd.cut(
    x=dfs['beta_sim'],
    bins=bins,
    include_lowest=True
)

dfs['beta_sim_mid'] = (
    dfs['beta_sim_int']
        .map(lambda x: x.mid)
)

dfd = (
    dfs['beta_sim_mid']
        .value_counts(normalize=True)
        .reset_index()
        .sort_values(by=['beta_sim_mid'], ascending=True)
        .reset_index(drop=True)
        .copy()
)

fig, ax = plt.subplots()
ax.bar(
    x=dfd['beta_sim_mid'],
    height=dfd['proportion'],
    width=0.8 * binwidth,
    label='Simulated Bootstrap Distribution'
)
ax.axvline(
    x=point_estimate,
    c='red',
    linewidth=5,
    label=f'Point estimate: {point_estimate:.1f}'
)
ax.set_ylabel('Relative Frequency')
ax.set_xlabel('Coefficient Value')
ax.legend(
    bbox_to_anchor=(0.5,-0.5),
    loc='center'
)
```

```
print(
    '\n1% Quantile:',
    f"{dfs['beta_sim'].quantile(0.01):.1f}",
    '\n2.5% Quantile:',
    f"{dfs['beta_sim'].quantile(0.025):.1f}",
    '\n5% Quantile:',
    f"{dfs['beta_sim'].quantile(0.05):.1f}",
    '\n95% Quantile:',
    f"{dfs['beta_sim'].quantile(0.95):.1f}",
    '\n97.5% Quantile:',
    f"{dfs['beta_sim'].quantile(0.975):.1f}",
    '\n99% Quantile:',
    f"{dfs['beta_sim'].quantile(0.99):.1f}",
    sep=''
)
```

```
1% Quantile:-84.4
2.5% Quantile:-82.6
5% Quantile:-80.9
95% Quantile:-63.6
97.5% Quantile:-62.0
99% Quantile:-60.1
```


0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
Relative Frequency
−90
−85
−80
−75
−70
−65
−60
−55
Coefficient Value
Point estimate: -72.3
Simulated Bootstrap Distribution